

TRENTOS-M Provisioning Tools

- [Overview](#)
 - [Roadmap](#)
 - [KeyStore Provisioning Tool](#)
 - [Minimum required effort to bring it in for 1.1](#)
 - [Possible improvements for 1.x](#)
 - [Configuration Provisioning Tool](#)
 - [Minimum required effort to bring it in for 1.0](#)
 - [Possible improvements for 1.x](#)
-

Overview

The current intention behind this page is to mainly break down the work that still needs to be done to bring in the provisioning tools for TRENTOS systems. In the long run this page could eventually become the starting point for the SDK documentation of the provisioning tools.

Roadmap

The following listing of tickets should provide a better overview of what still needs to be done to have the Provisioning Tools in the SDK. It should be emphasized that the listed tickets assume that the main priority is to have the tools working and producing utilizable images. Any additional beautification or improvements were not considered as bare necessities for the 1.0 release.

All listed tickets are sorted in steps that reflect the order in which they should be executed due to their inherent dependencies.

KeyStore Provisioning Tool

Current state of the implementation:

- The tool can parse XML files that contain elements formatted in the following style:

```

<!--
KEY TYPE:
    AES key           = 0
    RSA key pair      = 1
    DH key pair       = 2
    SECP256R1 key pair = 3

KEY NAME:
    String representing key name

KEY EXPORTABLE:
    Key can be exported (only meaningful if key is used in isolated component)

KEY LENGTH:
    Number in bits

KEY BYTES:
    String representing raw key = Import key
    0                          = Generate key

-->

<key>
  <type>0</type>
  <name>AESKeyImported</name>
  <exportable>1</exportable>
  <length>256</length>
  <bytes>leoZwkPoYEGUWxJug9v2U5oE6GiIim1R</bytes>
</key>
<key>
  <type>0</type>
  <name>AESKeyGenerated</name>
  <exportable>1</exportable>
  <length>256</length>
  <bytes>0</bytes>
</key>
<key>

```

- It currently supports parsing and storing AES keys and KeyPairs (RSA, DH...).
- The whole XML parsing logic is done by a python script and the script then calls the tool with varying cmd line parameters to write the different keys. The tool itself does not contain any XML parsing logic and can only process one key per run of the tool.
- The tool stores all keys to the same KeyStore instance name, which could be used to store the keys under specific client "domains".

Minimum required effort to bring it in for 1.1

- Is the current way it is integrated in the SDK matching the expectation?
- Should this tool also be adapted to not use python to parse the XML file and call the tool but rather have all the XML parsing logic inside the tool as it was prepared for the configuration tool? [SEOS-1385 - Getting issue details...](#) STATUS
- Should it be showcased in a demo?

Step	Ticket	Comment	Preliminary StoryPoint Estimation
1	SEOS-1090 - Getting issue details... STATUS	Reactivate the demo and the PyTest scripts for it and see if the tool in its current state creates a usable image. This will include bringing in new features such as the generic UART and ChanMux component to make it run properly with our current SDK.	5-8
2	SEOS-1385 - Getting issue details... STATUS	Rework the KeyStore Provisioning Tool to include the XML parsing logic. The current implementation of the KeyStore Provisioning Tool utilizes a Python script to parse the XML file and call the tool repeatedly with the parsed key parameters.	8
3	SEOS-1008 - Getting issue details... STATUS	SDK Documentation for Key Store Provisioning Tool	3

4	SEOS-1117 - Getting issue details... STATUS	Improve code quality.	2
---	--	-----------------------	---

Possible improvements for 1.x

Step	Ticket	Comment	Preliminary StoryPoint Estimation
1	SEOS-1087 - Getting issue details... STATUS	The keystore provisioning tool currently uses a dummy function for entropy collection, this must be connected to the system RNG, otherwise all generated keys are predictable.	?
2	Decision needed here if and what of this should be done for the SDK	Either extend the SDK with a demo system using the KeyStore component or extend the IoT Demo to make use of the KeyStore so the usage of the Provisioning Tool can be showcased in the SDK.	?

Configuration Provisioning Tool

Minimum required effort to bring it in for 1.0

[SEOS-794](#) - Getting issue details... STATUS

The underlying goal here would be to make it work, integrate it in the SDK build flow and find an acceptable way to showcase it in the IoT demo. Guiding idea of the minimum required effort here would really be to minimize any "beautification" tasks.

Step	Ticket	Comment	Preliminary StoryPoint Estimation
1	SEOS-1374 - Getting issue details... STATUS	Trying to build the current state of the tool during the work on SEOS-1308 - Getting issue details... STATUS uncovered that as expected the changes in the SDK affect the build process of the tool. A first analysis showed that this is not only related to renaming of functions and error codes but also that the tool is referring to files that do not exist or at least were never pushed to the repository of the tool. The effects here might not be dramatic and can be easily recovered but it this will need to be further investigated.	8
2	SEOS-1337 - Getting issue details... STATUS	The IoT demo or the test_config_server_fs_backend can be quite easily used to verify this by removing the current component (Admin or ConfigFileInjector) that injects the configuration backend.	3
3	SEOS-1413 - Getting issue details... STATUS	Depending on the outcome of step 2, a follow up ticket might be created in case the tool does not produce a usable image and the output of the tool cannot be fixed easily within the scope of the ticket. As we currently do not know how much work is required here, we should make sure to have a reasonable buffer for this task.	13
4	SEOS-1077 - Getting issue details... STATUS	This story heavily depends on any additional review comments and the effort we want to put into making the code base acceptable for 1.0. The estimation given here is based on the review comment that have already been placed for the existing PullRequests.	5-8
5	SEOS-1340 - Getting issue details... STATUS	Estimation here depends heavily on whether or not this build flow can be set up similar to the KeyStore Provisioning Tool or if this is needs to be reworked altogether.	5
6	SEOS-1345 - Getting issue details... STATUS	Once the demo is reworked and in further consequence will be relying on the tool, the documentation of the demo for the release will need to be updated (this is also included in the DoD of the ticket). The CI scripts will also need to be updated so the tool creates the NVM file before the demo start running.	8
7	SEOS-1382 - Getting issue details... STATUS	Create SDK Documentation for Configuration Provisioning Tool	3

8	SEOS-1118 - Getting issue details... STATUS	Improve code quality.	2
---	--	-----------------------	---

Possible improvements for 1.x

Step	Ticket	Comment	Preliminary StoryPoint Estimation
1	SEOS-1347 - Getting issue details... STATUS	In regards to the time sensitive 1.0, this can be seen as a more optional step that will provide us with a test system that verifies the functionality of the tool in combination with a TRENTOS system. Basically this is already covered by the IoT demo with its tests in CI but should also in time be extended to this since I believe we don't want our demos to be the main tests for our code base.	5
2	SEOS-1383 - Getting issue details... STATUS		3